

PolyFlop8

Top-Level System Integration Test Card

TS-ST-TOP-001

Verification of Full CPU with Control Unit FSM

Test Plan ID:	TS-ST-TOP-001
Revision:	1.0
Date:	January 6, 2026
Test Stage:	Stage 4 - Top-Level System Integration
Author:	Artem Horiunov
Status:	APPROVED
Coverage Target:	85% System Coverage

Contents

1	Test Overview	2
1.1	Purpose	2
1.2	Scope	2
1.3	Key Subsystems Under Test	2
2	Test Cases	3
2.1	Test Case SYS-01: "Hello World" (Simple Add)	3
2.2	Test Case SYS-02: Memory Store & Load	4
2.3	Test Case SYS-03: Conditional Loop (BRNE)	5
2.4	Test Case SYS-04: I/O Output Test	6
2.5	Test Case SYS-05: Stack Pointer / Call	7

1 TEST OVERVIEW

1.1 Purpose

This document defines the test cases for Stage 4: Top-Level System Integration testing of the PolyFlop8 microcontroller.

1.2 Scope

- Verification of complete instruction cycle: Fetch → Decode → Execute → Write-Back
- Verification of Control Unit FSM state transitions and timing
- Verification of integrated memory subsystem (RAM access)
- Verification of I/O operations and peripheral communication
- Execution of complete assembly programs

1.3 Key Subsystems Under Test

Subsystem	Test Focus	Covered Requirements
Arithmetic Pipeline	ALU → RegFile → SREG → Control	TR-ALU-01/02, TR-ISA-01 to 07
Memory Subsystem	CU → RAM → RegFile → ALU	TR-RAM-01 to 05, TR-ISA-10 to 12
Control Flow	CU → PC → ROM → Decoder → CU	TR-CU-01 to 07, TR-ISA-13 to 17
I/O Operations	CU → I/O → RegFile → External	TR-IO-01 to 05, TR-ISA-19/20

2 TEST CASES

2.1 Test Case SYS-01: "Hello World" (Simple Add)

Test ID	SYS-01
Priority	P1
Title	"Hello World" (Simple Arithmetic Sequence)
Objective	Verify basic instruction fetch, decode, execute pipeline with arithmetic
Requirements	TR-CU-01, TR-CU-03, TR-CU-05, TR-ISA-01

Test Program:

```
.org 0x000
LDI R1, 0x02    ; Load immediate 2 into R1
LDI R2, 0x03    ; Load immediate 3 into R2
ADD R1, R2      ; R1 = R1 + R2 (2 + 3 = 5)
NOP            ; Wait for result
```

Pre-conditions	Reset applied. ROM loaded with test program. All registers = 0x00.
Test Steps	• Release reset ($rst = '0'$) • Run simulation for 100ns (10 clock cycles at 100MHz) • Monitor: PC sequence, register values, ALU result, flags
Expected Results	• Cycle 1: PC=0x000, Fetch LDI R1, 0x02 • Cycle 2: PC=0x001, Execute LDI → R1=0x02 • Cycle 3: PC=0x001, Fetch LDI R2, 0x03 • Cycle 4: PC=0x002, Execute LDI → R2=0x03 • Cycle 5: PC=0x002, Fetch ADD R1, R2 • Cycle 6: PC=0x003, Execute ADD → R1=0x05 • ALU result during ADD: 0x05
Pass Criteria	All instructions execute correctly with proper timing

2.2 Test Case SYS-02: Memory Store & Load

Test ID	SYS-02
Priority	P1
Title	Memory Store & Load (RAM Access Verification)
Objective	Verify complete memory subsystem: store to and load from RAM
Requirements	TR-RAM-01 to 05, TR-CU-04, TR-ISA-10 to 12

Test Program:

```
.org 0x000
LDI R1, 0xFF      ; Load test value
STS 0x05, R1      ; Store to address 0x05 (Direct)
CLR R1            ; Clear R1 (XOR R1, R1)
LDS R1, 0x05      ; Load from address 0x05
; Alternative indirect test
LDI R7, 0x10      ; Setup X-pointer
ST X, R1          ; Store indirect
CLR R1            ; Clear again
LD R1, X          ; Load indirect
```

Pre-conditions	Reset applied. ROM loaded with test program. RAM initialized to 0x00.
Test Steps	• Release reset and run simulation • Monitor RAM address 0x05 during store operation • Monitor R1 value throughout test • Verify timing: store takes 2 cycles, load takes 2 cycles • Test both direct (LDS/STS) and indirect (LD/ST) addressing
Expected Results	• After STS: RAM[0x05] = 0xFF • After CLR: R1 = 0x00 • After LDS: R1 = 0xFF (reloaded from RAM) • Indirect store: RAM[0x10] = 0xFF • Indirect load: R1 = 0xFF (from RAM[0x10]) • Memory control signals: mem_we='1' only during store
Pass Criteria	All memory operations execute correctly with proper timing

2.3 Test Case SYS-03: Conditional Loop (BRNE)

Test ID	SYS-03
Priority	P2
Title	Conditional Loop (BRNE Counter Implementation)
Objective	Verify conditional branching and loop control flow
Requirements	TR-CU-06, TR-ISA-14, TR-ISA-15, TR-ISA-07

Test Program:

```
.org 0x000
    LDI R1, 0x03    ; Initialize counter
Loop:
    DEC R1          ; Decrement counter
    BRNE Loop       ; Branch if R1 != 0
    NOP             ; Exit point
```

Pre-conditions	Reset applied. ROM loaded with test program.
Test Steps	• Release reset and run simulation • Monitor PC sequence and R1 value each cycle • Count loop iterations • Verify branch taken/not taken behavior
Expected Results	• Initial: R1 = 0x03 • Iteration 1: DEC → R1=0x02, Z=0, BRNE taken → PC loops • Iteration 2: DEC → R1=0x01, Z=0, BRNE taken → PC loops • Iteration 3: DEC → R1=0x00, Z=1, BRNE NOT taken → PC advances • Total cycles: 8
Pass Criteria	Loop executes exactly 3 times, exits correctly

2.4 Test Case SYS-04: I/O Output Test

Test ID	SYS-04
Priority	P2
Title	I/O Output Test (Peripheral Communication)
Objective	Verify I/O write operations to output ports

Test Program:

```
.org 0x000
LDI R1, 0xAA      ; Load test pattern 10101010
OUT 0x01, R1      ; Write to I/O port 0x01
LDI R1, 0x55      ; Load complementary pattern 01010101
OUT 0x01, R1      ; Write to same port
NOP               ; Wait
```

Expected Results	• After first OUT: port_out_a = 0xAA (10101010) • io_we='1' during OUT execution cycle • After second OUT: port_out_a = 0x55 (01010101) • Port retains 0x55 until next write or reset
Pass Criteria	I/O writes execute correctly with proper timing and addressing

2.5 Test Case SYS-05: Stack Pointer / Call

Test ID	SYS-05
Priority	P3
Title	Stack Pointer / Call (Subroutine Call and Return)
Objective	Verify subroutine call/return mechanism and stack operation

Test Program:

```
.org 0x000
LDI R7, 0xFF      ; Initialize stack pointer (R7 = X-pointer)
RCALL Subr        ; Call subroutine
NOP               ; Return here
HALT: RJMP HALT   ; Infinite loop

.org 0x020
Subr:
LDI R1, 0x55      ; Subroutine code
RET               ; Return
```

Expected Results	• Before RCALL: PC=0x002, R7=0xFF • During RCALL: PC jumps to 0x020 • Stack operation: RAM[0xFF] stores return address (0x003) • RET operation: PC loads return address from stack
Coverage Points	Subroutine linkage, stack pointer management, return address handling

APPENDIX A: COMPLETE TEST PROGRAM SUITE

```

1  ;=====
2  ; PolyFlop8 System Integration Test Suite
3  ;=====
4
5  .org 0x000
6  ; Test 1: Basic Arithmetic
7      LDI R1, 0x02
8      LDI R2, 0x03
9      ADD R1, R2      ; R1 = 5
10     SUB R1, R2      ; R1 = 2
11     AND R1, R2      ; R1 = 2 & 3 = 2
12     OR R1, R2       ; R1 = 2 | 3 = 3
13     XOR R1, R2      ; R1 = 3 ^ 3 = 0
14
15 ; Test 2: Memory Operations
16     LDI R1, 0xAA
17     STS 0x10, R1     ; Store to RAM
18     CLR R1
19     LDS R1, 0x10     ; Load from RAM
20     LDI R7, 0x20     ; Setup X-pointer
21     ST X, R1         ; Store indirect
22     LD R2, X         ; Load indirect
23
24 ; Test 3: Control Flow
25     LDI R3, 0x03
26 Loop:
27     DEC R3
28     BRNE Loop        ; Loop 3 times
29
30     LDI R4, 0x01
31     CPI R4, 0x01
32     BRBS 1, Equal    ; Branch if Z=1
33     NOP              ; Should not reach
34 Equal:
35     RJMP NextTest
36
37 ; Test 4: I/O Operations
38 NextTest:
39     LDI R5, 0x55
40     OUT 0x01, R5      ; Write to output port
41     IN R6, 0x00       ; Read from input port
42
43 ; Test 5: Subroutine Call
44     RCALL Subroutine
45     NOP              ; Return here
46
47 ; Test 6: Complex Arithmetic
48     LDI R1, 0x80     ; Load min negative
49     NEG R1           ; Negate
50     RJMP Finish
51
52 Subroutine:
53     LDI R0, 0xFF
54     RET
55
56 Finish:
57     RJMP Finish      ; Infinite Loop

```

Listing 1: Master System Test Program